

Graphic Paint

Módulo de Curvas Paramétricas

Curvas de Bézier e B-*spline* Cúbica

Fernando Campos Silva Dal' Maria

Junho de 2026

Repositório: [/Graphic-Paint](#)

Versão hospedada: [Paint](#)

Sumário

1	Visão Geral	2
2	Organização do Código	2
2.1	Estrutura de pastas relevante	2
2.2	Hierarquia de classes	3
2.3	Arquivo: <code>_shapes/BezierCurve.ts</code>	3
2.4	Arquivo: <code>_shapes/BSplineCurve.ts</code>	3
2.5	Arquivo: <code>_hooks/useCurveDrawing.ts</code>	4
2.6	Modificações em arquivos existentes	4
2.6.1	<code>_types/Graphics.ts</code>	4
2.6.2	<code>_hooks/useDrawingHandlers.ts</code>	5
2.6.3	<code>_utils/scenePersistence.ts</code>	5
2.6.4	<code>_components/menu/main/ui/ShapeSelector.tsx</code>	6
3	Modelo Matemático	6
3.1	Curva de Bézier	6
3.1.1	Definição via polinômios de Bernstein	6
3.1.2	Algoritmo de De Casteljau	6
3.1.3	Casos especiais na renderização vetorial	7
3.1.4	Renderização pixelada	7
3.2	B-spline Cúbica com Nós <i>Clamped</i>	8
3.2.1	Funções base de B-spline (Cox-de Boor)	8
3.2.2	Vetor de nós <i>clamped</i> uniforme	8
3.2.3	Algoritmo de de Boor	9
3.2.4	Comparativo: Bézier vs. B-spline	10
4	Manual de Uso	10
4.1	Selecionando a ferramenta	10
4.2	Desenhando uma curva de Bézier	10
4.3	Desenhando uma B-spline cúbica	11
4.4	Propriedades comuns às duas ferramentas	12
5	Referências	13

1 Visão Geral

Este documento descreve a implementação e o uso das ferramentas de **curvas paramétricas** adicionadas ao módulo de pintura (*paint*) do aplicativo **Graphic Paint**. Foram integradas duas curvas clássicas de design geométrico:

- **Curva de Bézier** de grau arbitrário, avaliada pelo algoritmo de De Casteljau [1].
- **B-spline cúbica uniforme** com vetor de nós *clamped*, avaliada pelo algoritmo de de Boor [2].

Ambas seguem exatamente a arquitetura de formas já existente no projeto: herdam da classe abstrata **Shape**, são serializáveis/desserializáveis, suportam o modo pixelado e entram no fluxo de *pending-placement* (mover, rotacionar e redimensionar antes de confirmar).

2 Organização do Código

2.1 Estrutura de pastas relevante

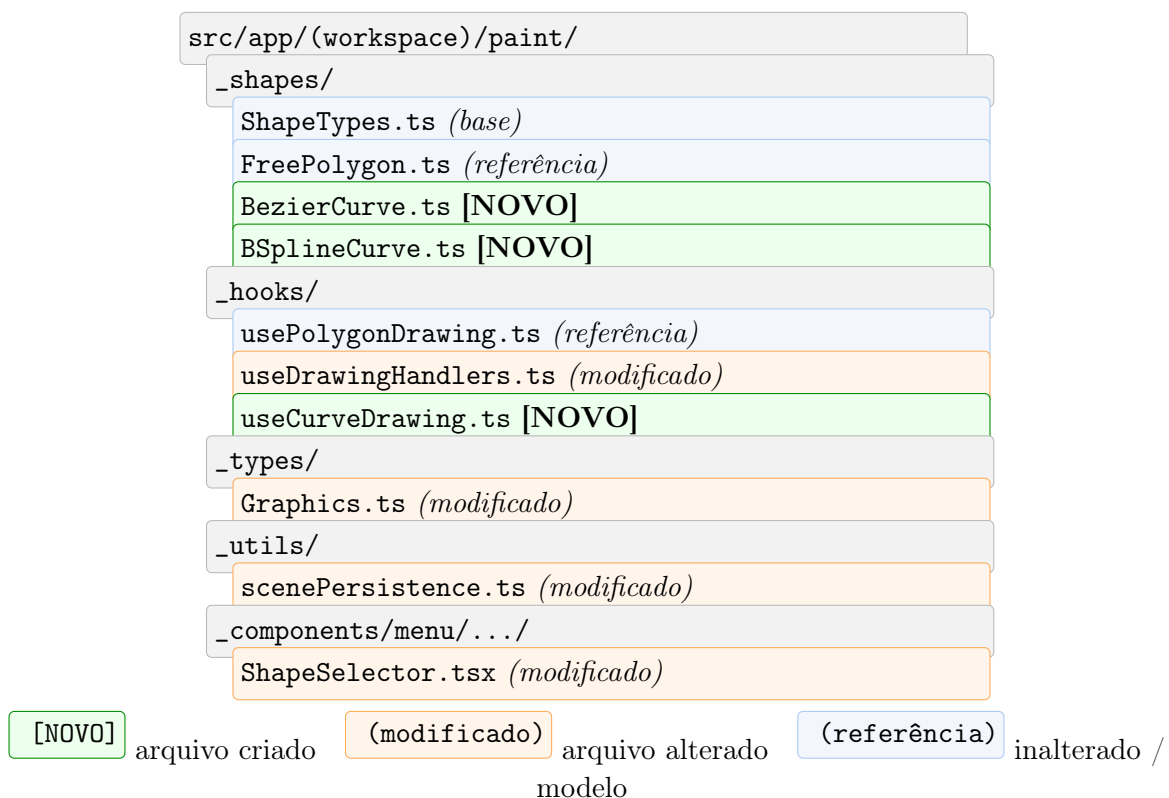


Figura 1: Arquivos criados (verde) e modificados (laranja) na implementação.

2.2 Hierarquia de classes

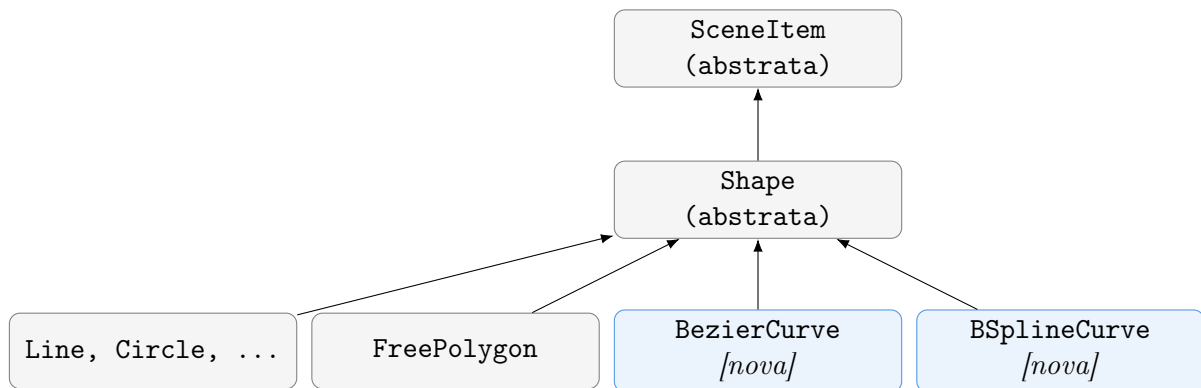


Figura 2: Hierarquia de herança. Tanto `BezierCurve` quanto `BSplineCurve` herdam de `Shape` diretamente, seguindo o mesmo padrão de todas as demais formas geométricas.

2.3 Arquivo: `_shapes/BezierCurve.ts`

Implementa a curva de Bézier de grau arbitrário.

Símbolo	Responsabilidade
<code>deCasteljau(pts, t)</code>	Avalia um único ponto $B(t)$ via interpolações lineares sucessivas sobre uma cópia temporária dos pontos de controle.
<code>sampleBezier(pts, steps)</code>	Gera um array de <code>steps+1</code> pontos amostrando t uniformemente em $[0, 1]$.
<code>BezierCurve.standardDraw()</code>	Renderização vetorial: usa API nativa do Canvas (<code>quadraticCurveTo</code> / <code>bezierCurveTo</code>) para graus 2–3; para grau ≥ 4 usa amostragem + <code>lineTo</code> .
<code>BezierCurve.pixelatedDraw()</code>	Amostragem em coordenadas de grade $\rightarrow$ Bresenham entre pares consecutivos $\rightarrow$ <code>drawPixel</code> .
<code>BezierCurve.getBoundingBox()</code>	Envelope AABB dos pontos de controle (conservador, mas suficiente para transformações).
<code>BezierCurve.moveBy()</code>	Delega a <code>translatePointCollection()</code> .
<code>BezierCurve.rotateBy()</code>	Delega a <code>rotatePoints()</code> , utilizando snapshot congelado para evitar deriva de arredondamento.
<code>BezierCurve.resizeToBoundingBox()</code>	Delega a <code>resizePointCollection()</code> .

Tabela 1: Responsabilidades dos símbolos em `BezierCurve.ts`.

2.4 Arquivo: `_shapes/BSplineCurve.ts`

Implementa a B-spline cúbica uniforme com nós *clamped*.

Símbolo	Responsabilidade
<code>buildClampedKnots(n, k)</code>	Constrói o vetor de nós T de comprimento $n + k + 1$ com multiplicidade $k + 1$ nos extremos.
<code>findKnotSpan(n, k, t, T)</code>	Busca binária pelo índice do span i tal que $T_i \leq t < T_{i+1}$; trata o extremo superior.
<code>deBoor(pts, T, k, span, t)</code>	Triângulo de de Boor: k passos de interpolação linear, retorna $C(t)$.
<code>sampleBSpline(pts, steps)</code>	Amostra uniformemente o domínio $[T_k, T_n]$ em <code>steps+1</code> pontos.
<code>BSplineCurve.standardDraw()</code>	Renderização vetorial por amostragem + <code>lineTo</code> .
<code>BSplineCurve.pixelatedDraw()</code>	Amostragem em grade $\rightarrow$ Bresenham $\rightarrow$ <code>drawPixel</code> .

Tabela 2: Responsabilidades dos símbolos em `BSplineCurve.ts`.

2.5 Arquivo: `_hooks/useCurveDrawing.ts`

React Hook que gerencia o estado de interação multi-clique compartilhado pelas duas ferramentas de curva.

Símbolo / constante	Responsabilidade
<code>points</code> (ref)	Acumula os pontos de controle já confirmados.
<code>cursor</code> (ref)	Posição atual do ponteiro (pré-confirmação).
<code>lastClickTime</code> (ref)	Instante do último clique, para detecção de duplo-clique ($\Delta t < 300$ ms).
<code>BEZIER_MIN_POINTS = 3</code>	Mínimo para emitir uma curva de Bézier válida.
<code>BSPLINE_MIN_POINTS = 4</code>	Mínimo para a B-spline cúbica <i>clamped</i> .
<code>drawPreview()</code>	Redesenha o canvas com: polígono de controle tracejado, marcadores nos pontos confirmados e a curva resultante (se pontos suficientes).
<code>finalize()</code>	Cria a instância final da curva e chama <code>enterPending()</code> para o fluxo de placement.
<code>cancel()</code>	Descarta todos os pontos e redesenha a cena.
<code>onPointerDown()</code>	Adiciona ponto ou finaliza em duplo-clique.
<code>onPointerMove()</code>	Atualiza o cursor e redesenha o preview.

Tabela 3: Símbolos relevantes em `useCurveDrawing.ts`.

2.6 Modificações em arquivos existentes

2.6.1 `_types/Graphics.ts`

O tipo de união `Geometric` recebeu dois novos membros:

```

1 export type Geometric =
2   // ... membros anteriores ...
3   | 'bezier' // curva de Bezier
4   | 'bspline'; // B-spline cubica clamped

```

Listing 1: Adição ao tipo `Geometric`.

2.6.2 _hooks/useDrawingHandlers.ts

Foram feitas três alterações pontuais:

1. **Import:** import useCurveDrawing from './useCurveDrawing'
2. **Instância do hook:** chamada a useCurveDrawing({...}) logo após a instância de usePolygonDrawing.
3. **Despacho de eventos:** nas callbacks handlePointerDown e handlePointerMove, adicionado o bloco:

```

} else if (selectedShape === 'bezier' || selectedShape ===
  'bspline') {
  curve.onPointerDown(mappedPoint); // ou onPointerMove
}

```

Listing 2: Despacho para o hook de curvas.

e em handlePointerUp a guarda:

```

const isCurveShape = selectedShape === 'bezier' ||
  selectedShape === 'bspline';
if (!pendingPointerUp() && selectedShape !== 'polygon'
  && !isCurveShape && isDrawing.current) { ... }

```

2.6.3 _utils/scenePersistence.ts

Adicionados dois imports, dois casos em serializeSceneItem() e dois casos em deserializeSceneItem(). A estrutura serializada é idêntica à do polígono livre, apenas um array de pontos:

```

1 // Serializacao (serializeSceneItem)
2 if (item instanceof BezierCurve || item instanceof
  BSplineCurve) {
3   return { kind: item.kind, data: {
4     ...getCommonShapeData(item),
5     points: clonePoints(item.points) } };
6 }
7 // Desserializacao (switch em deserializeSceneItem)
8 case 'bezier': {
9   const shape = new BezierCurve(readPoints(data.points),
10    getShapeOptions(data));
11   applyCommonShapeData(shape, data);
12   return shape;
13 }
14 case 'bspline': {
15   const shape = new BSplineCurve(readPoints(data.points),
16    getShapeOptions(data));
17   applyCommonShapeData(shape, data);
18   return shape;
19 }

```

Listing 3: Padrão de serialização das curvas.

2.6.4 _components/menu/main/ui/ShapeSelector.tsx

Importados TbVectorBezier2 e TbVectorSpline do pacote react-icons/tb, e adicionados dois WorkspaceToolButton na barra inferior do seletor de formas.

3 Modelo Matemático

3.1 Curva de Bézier

3.1.1 Definição via polinômios de Bernstein

Definição 3.1 (Curva de Bézier de grau n). *Dados $n+1$ pontos de controle $\mathbf{P}_0, \mathbf{P}_1, \dots, \mathbf{P}_n \in \mathbb{R}^2$ e um parâmetro $t \in [0, 1]$, a curva de Bézier de grau n é o mapa:*

$$\mathbf{B}(t) = \sum_{i=0}^n \binom{n}{i} (1-t)^{n-i} t^i \mathbf{P}_i = \sum_{i=0}^n B_{i,n}(t) \mathbf{P}_i, \quad (1)$$

onde $B_{i,n}(t) = \binom{n}{i} (1-t)^{n-i} t^i$ é o i -ésimo polinômio de Bernstein de grau n .

Os polinômios de Bernstein satisfazem a partição da unidade $\sum_{i=0}^n B_{i,n}(t) = 1$ e são todos não-negativos em $[0, 1]$, o que garante que a curva reside no *invólucro convexo* (casco convexo) dos pontos de controle.

Propriedade 3.1 (Interpolação nas extremidades). $\mathbf{B}(0) = \mathbf{P}_0$ e $\mathbf{B}(1) = \mathbf{P}_n$.

Propriedade 3.2 (Tangente nas extremidades).

$$\mathbf{B}'(0) = n(\mathbf{P}_1 - \mathbf{P}_0), \quad \mathbf{B}'(1) = n(\mathbf{P}_n - \mathbf{P}_{n-1}).$$

A tangente de saída é paralela ao segmento $\mathbf{P}_0\mathbf{P}_1$ e a tangente de chegada é paralela a $\mathbf{P}_{n-1}\mathbf{P}_n$.

3.1.2 Algoritmo de De Casteljau

A avaliação direta de (1) é numericamente instável para graus elevados. O algoritmo de De Casteljau [1] computa o mesmo resultado por subdivisão recursiva:

Definição 3.2 (Algoritmo de De Casteljau). *Define-se a família de pontos intermediários:*

$$\mathbf{P}_i^{(0)} = \mathbf{P}_i, \quad i = 0, \dots, n, \quad (2)$$

$$\mathbf{P}_i^{(r)} = (1-t)\mathbf{P}_i^{(r-1)} + t\mathbf{P}_{i+1}^{(r-1)}, \quad r = 1, \dots, n, \quad i = 0, \dots, n-r. \quad (3)$$

O ponto da curva é $\mathbf{B}(t) = \mathbf{P}_0^{(n)}$.

O processo pode ser visualizado como um triângulo de interpolações lineares sucessivas (Tabela 4).

$r = 0$	$r = 1$	$r = 2$	$r = 3$	$\dots$	$r = n$
$P_0^{(0)}$	$P_0^{(1)}$	$P_0^{(2)}$	$P_0^{(3)}$	$\dots$	$B(t)$
$P_1^{(0)}$	$P_1^{(1)}$	$P_1^{(2)}$	$P_1^{(3)}$		
$P_2^{(0)}$	$P_2^{(1)}$	$P_2^{(2)}$			
$P_3^{(0)}$	$P_3^{(1)}$				
$\vdots$					

Tabela 4: Triângulo de De Casteljau para grau n . Cada nível r é calculado interpolando o nível $r - 1$.

Pseudocódigo.

```

1 function deCasteljau(P[0..n], t):
2   d[0..n] <- copia de P[0..n]           // buffer temporario
3   Para r de 1 ate n:
4     Para i de 0 ate n - r:
5       d[i].x <- (1-t)*d[i].x + t*d[i+1].x
6       d[i].y <- (1-t)*d[i].y + t*d[i+1].y
7   Retorna d[0]
```

Listing 4: De Casteljau, avaliação de $B(t)$.

Complexidade. $O(n^2)$ operações de ponto flutuante por avaliação. Para graus práticos ($n \leq 20$) e o número de amostras utilizado (≤ 400), o custo é negligenciável em hardware moderno.

3.1.3 Casos especiais na renderização vetorial

A implementação aproveita as primitivas nativas do Canvas 2D para os graus mais comuns, eliminando o custo de amostragem:

Grau n	Nº de CPs	Primitiva usada
1	2	<code>ctx.lineTo(P₁)</code>
2	3	<code>ctx.quadraticCurveTo(P₁, P₂)</code>
3	4	<code>ctx.bezierCurveTo(P₁, P₂, P₃)</code>
≥ 4	≥ 5	De Casteljau + $\max(100, n \times 30)$ <code>lineTo</code>

Tabela 5: Mapeamento grau $\rightarrow$ primitiva Canvas 2D.

3.1.4 Renderização pixelada

Em modo pixelado, a curva é amostrada em coordenadas de grade (*pixels* lógicos de tamanho `pixelSize`) e cada segmento entre amostras consecutivas é rasterizado pelo algoritmo de Bresenham. O número de amostras é dimensionado pela diagonal da *bounding box*:

$$\text{steps} = \max\left(3 \cdot \left\lceil \frac{\sqrt{w^2 + h^2}}{\text{pixelSize}} \right\rceil, 20 \cdot n, 50\right), \quad (4)$$

onde w e h são a largura e a altura da *bounding box* em coordenadas do documento. Isso garante que não haja lacunas na curva rasterizada, mesmo quando os pontos de controle cobrem grande área.

3.2 B-spline Cúbica com Nós *Clamped*

3.2.1 Funções base de B-spline (Cox-de Boor)

Definição 3.3 (Vetor de nós). *Um vetor de nós é uma sequência não-decrescente $T = (t_0 \leq t_1 \leq \dots \leq t_m)$. Para uma B-spline de grau k com n pontos de controle, $m = n + k + 1$ (i.e., $m + 1$ nós).*

Definição 3.4 (Funções base, recursão de Cox-de Boor [3, 2]).

$$N_{i,0}(t) = \begin{cases} 1 & \text{se } t_i \leq t < t_{i+1}, \\ 0 & \text{caso contrário,} \end{cases} \quad (5)$$

$$N_{i,k}(t) = \frac{t - t_i}{t_{i+k} - t_i} N_{i,k-1}(t) + \frac{t_{i+k+1} - t}{t_{i+k+1} - t_{i+1}} N_{i+1,k-1}(t), \quad (6)$$

com a convenção $0/0 = 0$.

A curva B-spline é então:

$$\mathbf{C}(t) = \sum_{i=0}^{n-1} N_{i,k}(t) \mathbf{P}_i, \quad t \in [t_k, t_n]. \quad (7)$$

3.2.2 Vetor de nós *clamped* uniforme

Para que a curva *interpole* os pontos inicial e final, usa-se o vetor de nós *clamped* (com multiplicidade $k + 1$ nos extremos):

$$T = \underbrace{0, \dots, 0}_{k+1}, 1, 2, \dots, n - k - 1, \underbrace{n - k, \dots, n - k}_{k+1}. \quad (8)$$

Para a B-spline cúbica ($k = 3$) com n pontos de controle ($n \geq 4$):

$$T = \underbrace{0, 0, 0, 0}_4, 1, 2, \dots, n - 4, \underbrace{n - 3, n - 3, n - 3, n - 3}_4. \quad (9)$$

Propriedade 3.3 (Interpolação nas extremidades). *Com o vetor clamped, $\mathbf{C}(0) = \mathbf{P}_0$ e $\mathbf{C}(n - 3) = \mathbf{P}_{n-1}$.*

Propriedade 3.4 (Continuidade C^2). *Em cada nó interno simples, a curva possui continuidade de segunda derivada (C^2), garantindo transições suaves entre segmentos cúbicos consecutivos.*

Propriedade 3.5 (Controle local). *Mover o ponto $\mathbf{P}_i$ afeta apenas os $k + 1 = 4$ segmentos que contêm $\mathbf{P}_i$ em sua janela de influência. Isso contrasta com a curva de Bézier, cujo controle é global.*

n	# nós	Vetor de nós T	Domínio	# segmentos
4	8	$[0, 0, 0, 0, 1, 1, 1, 1]$	$[0, 1]$	1
5	9	$[0, 0, 0, 0, 1, 2, 2, 2, 2]$	$[0, 2]$	2
6	10	$[0, 0, 0, 0, 1, 2, 3, 3, 3, 3]$	$[0, 3]$	3
n	$n + 4$	(fórmula 8)	$[0, n - 3]$	$n - 3$

Tabela 6: Exemplos do vetor de nós *clamped* para diferentes valores de n .

3.2.3 Algoritmo de de Boor

Para avaliar $C(t)$, localiza-se o *span* $[t_i, t_{i+1})$ que contém t e executa-se o algoritmo de de Boor [2], análogo ao triângulo de De Casteljau:

```

1 // Pre: span = i tal que T[i] <= t < T[i+1]
2 function deBoor(P[0..n-1], T[0..n+k], k, span, t):
3     d[0..k] <- P[span-k .. span] // janela de k+1 pontos de
4     controle
5
6     Para r de 1 ate k:
7         Para j de k ate r:
8             i_local <- span - k + j
9             denom <- T[i_local + k - r + 1] - T[i_local]
10            Se denom = 0:
11                alpha <- 0
12            Senao:
13                alpha <- (t - T[i_local]) / denom
14            d[j] <- (1 - alpha)*d[j-1] + alpha*d[j]
15
16 Retorna d[k]
```

Listing 5: Algoritmo de de Boor, avaliação de $C(t)$.

Localização do span. A implementação usa busca binária em $[T_k, T_n)$ com tratamento especial do extremo superior ($t = T_n$, que pertence ao último span não-vazio):

```

1 function findKnotSpan(n, k, t, T):
2     Se t >= T[n]:
3         i <- n - 1
4         Enquanto i > k E T[i] = T[n]:
5             i <- i - 1
6         Retorna i
7
8     lo <- k; hi <- n; mid <- (lo + hi) / 2
9     Enquanto t < T[mid] OU t >= T[mid+1]:
10        Se t < T[mid]:
11            hi <- mid
12        Senao:
13            lo <- mid
14        mid <- (lo + hi) / 2
15    Retorna mid
```

Listing 6: Localização do span por busca binária.

3.2.4 Comparativo: Bézier vs. B-spline

Característica	Bézier	B-spline cúbica
Grau da curva	$n = \#CPs - 1$	3 (fixo)
Nº mín. de pontos de controle	3	4
Interpolação de extremos	Sim (P_0, P_n)	Sim (<i>clamped</i>)
Controle de influência	Global	Local ($k + 1$ segmentos)
Continuidade	C^∞ (polinomial)	C^2 em nós simples
Adicionar ponto sem mudar grau	Não	Sim
Adequado para curvas longas	Grau alto $\Rightarrow$ oscilação	Sim, n arbitrário
Custo de avaliação	$O(n^2)$	$O(k^2) = O(9)$ por ponto

Tabela 7: Comparativo entre as duas curvas implementadas.

4 Manual de Uso

4.1 Selecionando a ferramenta

As duas ferramentas de curva ficam na barra inferior do **Seletor de Formas**, ao lado do botão de Polígono Livre:

Componente	Ferramenta	Tipo interno
TbVectorBezier2	Curva de Bézier	bezier
TbVectorSpline	B-spline cúbica	bspline

Tabela 8: Botões de seleção das ferramentas de curva no seletor de formas.

Observação 4.1. Clicar em uma ferramenta já ativa volta para a ferramenta de forma livre (**freeform**), comportamento padrão de todas as ferramentas do seletor.

4.2 Desenhando uma curva de Bézier

Passo a passo

Passo 1. Selecione a ferramenta Bézier clicando no botão TbVectorBezier2 no seletor de formas.

Passo 2. Adicione o primeiro ponto de controle clicando em qualquer posição do canvas. O ponto aparece como um marcador cheio.

Passo 3. Continue adicionando pontos com cliques únicos. A cada clique:

- Um marcador circular é desenhado no ponto confirmado.
- O polígono de controle (linhas tracejadas ligando todos os pontos) é atualizado.
- Assim que 3 ou mais pontos existirem, a *curva resultante* é exibida em tempo real.

Passo 4. Mova o mouse após cada clique para visualizar como o próximo ponto influenciará a curva (o cursor é mostrado como um anel aberto).

Passo 5. Finalize a curva com um dos métodos:

- **Duplo-clique**, o ponto do segundo clique NÃO é adicionado (o clique duplo finaliza sem novo ponto).
- **Tecla Enter**, finaliza com os pontos atuais.

Passo 6. Modo *pending-placement*: após finalizar, a curva entra no modo de posicionamento. É possível:

- **Mover** arrastando a curva.
- **Rotacionar** usando o puxador de rotação.
- **Redimensionar** pelos puxadores de canto/aresta.
- **Confirmar** clicando fora da curva ou pressionando **Enter**.
- **Cancelar** com **Escape**.

Cancelar durante a edição

Pressione **Escape** a qualquer momento para descartar todos os pontos adicionados e retornar ao estado inicial da ferramenta. Trocar de ferramenta também cancela automaticamente.

Quantidade de pontos e grau resultante

Nº de pontos clicados	Grau da curva	Primitiva usada
2	1 (reta)	<code>lineTo</code>
3	2 (quadrática)	<code>quadraticCurveTo</code>
4	3 (cúbica)	<code>bezierCurveTo</code>
5+	$n - 1$ (alto)	amostragem + <code>lineTo</code>

Tabela 9: Relação entre quantidade de cliques e grau da curva de Bézier.

4.3 Desenhando uma B-spline cúbica

Passo a passo

Passo 1. Selecione a ferramenta **B-spline** clicando em `TbVectorSpline`.

Passo 2. Adicione ao menos 4 pontos de controle com cliques únicos. Com menos de 4 pontos, o preview mostra apenas o polígono de controle sem a curva.

Passo 3. Continue adicionando pontos para refinar a forma. Cada novo ponto adiciona um segmento cúbico à curva. A propriedade de *controle local* significa que deslocar mentalmente um ponto afeta apenas os ≈ 4 segmentos próximos.

Passo 4. Finalize com duplo-clique ou **Enter** (mesmo comportamento da ferramenta Bézier).

Passo 5. Modo *pending-placement*: idêntico ao da Bézier.

Quantidade de pontos e segmentos

Nº de pontos (n)	Nº de segmentos cúbicos	Obs.
< 4	,	Curva não é exibida
4	1	1 cúbica (similar a Bézier cúbica)
5	2	C^2 na junção
6	3	C^2 em 2 junções
n	$n - 3$	C^2 em $n - 4$ nós internos

Tabela 10: Relação entre pontos de controle e segmentos da B-spline cúbica.

4.4 Propriedades comuns às duas ferramentas

Modo pixelado

Quando o modo pixelado está ativo (seletor de estilo de pintura), as curvas são rasterizadas usando o algoritmo de Bresenham entre amostras consecutivas, respeitando a grade de `pixelSize` unidades. O resultado é uma curva suave em termos de forma, mas com textura pixelada.

Espessura e cor

Ambas as ferramentas respeitam:

- **Cor atual** do seletor de cor.
- **Espessura** do controle de espessura (*thickness*).
- **Estilo de traço** (*dash preset*), sólido, tracejado, etc.
- **Algoritmo de linha** (*Bresenham* ou *DDA*) para o polígono de controle no modo pixelado.

Desfazer/Refazer

O botão de desfazer (*Undo*) funciona normalmente: após confirmar a curva no modo *pending-placement*, ela é adicionada à cena e pode ser desfeita junto com qualquer outra forma.

Seleção e transformações pós-commit

Após confirmada, a curva pode ser selecionada pela ferramenta de seleção (clique sobre ela) e:

- **Movida** arrastando.
- **Rotacionada** pelo puxador de rotação.
- **Redimensionada** pelos puxadores, os pontos de controle são remapeados proporcionalmente para a nova *bounding box*.
- **Copiada/Colada** pelo clipboard interno.
- **Salva** junto com o projeto (serialização em JSON via Supabase).

Diferenças práticas de uso

- Use a **Bézier** quando quiser uma única curva suave de forma previsível, ideal para lettering, formas orgânicas simples e traços de precisão com poucos pontos.
- Use a **B-spline** quando quiser uma curva composta de muitos pontos com controle suave entre eles, ideal para trajetórias longas, contornos de objetos complexos e refinamento incremental (adicionando pontos sem afetar outras partes da curva).

5 Referências

Referências

- [1] de Casteljaeu, P. (1959). *Outillages methodes calcul*. André Citroën Automobiles SA, Paris. Nota técnica interna.
- [2] de Boor, C. (1972). On calculating with B-splines. *Journal of Approximation Theory*, 6(1), 50–62. [https://doi.org/10.1016/0021-9045\(72\)90080-9](https://doi.org/10.1016/0021-9045(72)90080-9)
- [3] Cox, M. G. (1972). The numerical evaluation of B-splines. *IMA Journal of Applied Mathematics*, 10(2), 134–149. <https://doi.org/10.1093/imamat/10.2.134>
- [4] Farin, G. (2002). *Curves and Surfaces for CAGD: A Practical Guide* (5th ed.). Morgan Kaufmann. ISBN 978-1-55860-737-8. [§5 – Bézier curves; §6 – Geometric continuity]
- [5] Piegl, L., & Tiller, W. (1997). *The NURBS Book* (2nd ed.). Springer. ISBN 978-3-540-61545-3. [Algorithm A2.1 – FindSpan; Algorithm A3.1 – CurvePoint; §9.1 – Clamped knot vector]
- [6] Bresenham, J. E. (1965). Algorithm for computer control of a digital plotter. *IBM Systems Journal*, 4(1), 25–30. <https://doi.org/10.1147/sj.41.0025>